Package Info Component — Hackathon Problem Statement

1. The problem

Developers constantly check, compare and choose packages. A simple question, like how two libraries differ or what could replace an old one, means visiting several sites, comparing numbers and filtering out noise. A raw registry response doesn't answer it.

2. The challenge

Build a standalone hyperDart component that understands natural-language npm package queries and helps a developer decide: what a package is, how two or three compare, and what could replace it.

At minimum, your component should handle:

- **Package details**, including the latest version
- **Comparison** of two or three packages
- **Similar packages**: alternatives to a given package

Apart from the basic requirements, you can add more features, like:

- **Use-case queries** that name no package, such as `best npm package for date formatting` or `library for parsing csv`: work out the use case, then find, rank and recommend suitable packages
- **Install cost and compatibility:** package size, TypeScript support, ESM support and the Node versions it needs
- **Version-aware queries** such as `express 4 vs 5`
- **Follow-up actions** like "Compare with…" or "See alternatives" without a new search

> **What you show, how you show it, and how users interact with it are up to you. We're looking for original ideas, a unique design and solid engineering, not a copy of the npmjs.com package page. The best components answer the question at a glance and then let people dig deeper.**

3. What we provide

Trigger keywords and regex

Add these to `hyperdart.config.js`. You may extend them. Package names come from the `NPM_PACKAGES` collection (collection type `HD_PACKAGES`), which holds the top 10,000 packages. To trigger on other kinds of input too (for example locations or words), add regexes using any collection from the Entities docs.

```
keywords: ['npm', 'package info', 'packages like', 'compare packages']
```

```
regex: [
  '(npm( packages?)?|package info) HD_PACKAGES.NPM_PACKAGES',
  '(latest|current) version of HD_PACKAGES.NPM_PACKAGES',
  'compare( npm)?( packages?)? HD_PACKAGES.NPM_PACKAGES(,| vs| versus| and)?
HD_PACKAGES.NPM_PACKAGES((,| vs| versus| and)? HD_PACKAGES.NPM_PACKAGES)?',
  '(npm )?(alternatives?( to| for)|packages? (like|similar to))
HD_PACKAGES.NPM_PACKAGES( npm)?'
]
```

searchData

Your component receives a `searchData` prop with the packages found in the query, as entities (`sourceID` is the npm name). It doesn't tell you the intent: work that out from `query`. Depending on the template it may arrive as an array or a single object. Example for `compare react and vue` (trimmed):

```
{
  "component": "test-package",
  "query": "compare react and vue",
  "entities": [
    {
      "word": "react",
      "sourceID": "react",
      "collectionName": "NPM_PACKAGES",
      "collectionType": "HD_PACKAGES",
      "entityInfo": { "repository_url": "https://github.com/react/react" },
      "relevanceScore": 99.84998
    },
    {
      "word": "vue",
      "sourceID": "vue",
      "collectionName": "NPM_PACKAGES",
      "collectionType": "HD_PACKAGES",
```

```
      "entityInfo": { "repository_url": "https://github.com/vuejs/core" },
      "relevanceScore": 97.35074
    }
  ],
  "keyword": [],
  "genericQuery": "compare HD_PACKAGES__NPM_PACKAGES and
HD_PACKAGES__NPM_PACKAGES",
  "queryRegex": "compare( npm)?( packages?)? HD_PACKAGES__NPM_PACKAGES(,| vs|
versus| and) HD_PACKAGES__NPM_PACKAGES((,| vs| versus| and)
HD_PACKAGES__NPM_PACKAGES)?",
  "unambiguous": true,
  "suggestedPlace": "sidebar"
}
```


Data sources

The public npm registry and download-counts APIs. No API key is needed, and they accept calls from the browser. You may use other sources too.

Data	Endpoint
Latest version metadata	`https://registry.npmjs.org/{package}/latest`
Full package record (versions, dates)	`https://registry.npmjs.org/{package}`
Weekly downloads	`https://api.npmjs.org/downloads/point/last-week/{package}`
Daily downloads, last month	`https://api.npmjs.org/downloads/range/last-month/{package}`
Search	`https://registry.npmjs.org/-/v1/search?text={text}&size=25`

These are free public services: test your calls early, and expect gaps, limits and failures.

4. Example queries

User query	Component shows
`npm express`	The details card for express
`latest version of react`	react's latest version and release date first, then key details

User query	Component shows
`compare packages express, fastify`	A side-by-side table, a downloads chart and factual takeaway lines
`compare express, fastify, koa`	The same for three packages
`npm alternatives to moment`	A ranked list of similar packages, each with its shared keywords and weekly downloads
`npm request`	Details with a clear deprecated badge and npm's deprecation message

5. Submission

1. Source code in a public GitHub repository. Include a `.gitignore` so `node_modules` and `dist` aren't committed (the templates don't ship one).
2. The component published to the hyperDart dev registry.
3. A README covering your idea, design decisions, data sources, libraries, and how you built it (including any AI tools you used and what you wrote or changed yourself).
4. A 2–3 minute demo video on YouTube.

Add both links to `hyperdart.config.js`, using the field names already in your template:

```
github: 'https://github.com/<user>/<repo>',     // repo root, not a
/tree/main link
demovideo: 'https://www.youtube.com/watch?v=<id>'  // or
https://youtu.be/<id>
```

Publish with plain `npm publish`. It runs the prepublish validation (`scripts/validate-publish.js`), which checks keywords, regex, build output and both links. **Submissions published with validation skipped (for example `npm publish --ignore-scripts`, or by removing the `prepublishOnly` script) won't be evaluated.**

Before your first publish, create the package in the Partner Portal (My packages → Create package) and set your token with the command given in portal.

6. Judging criteria

Criterion	Weight	What judges look for
Design and creativity	25%	An original, unique design and interaction that works in sidebar and mainline, light and dark

Criterion	Weight	What judges look for
Usefulness and clarity	20%	How quickly the component answers the user's actual question
Accuracy and responsibility	15%	Faithful use of npm data: correct numbers, no misleading charts, no invented "winner"
Robustness	15%	Behaviour across different packages and queries, API failures, rate limits and unknown packages
Understanding (live review)	15%	Explaining the code and making a small change live, for example adding a query pattern
Code quality and README	10%	Readable code and a clear README

Extra features can lift a score, but only once the basic requirements work.

7. Resources

Resource	Link
hyperDart developer docs	hyperdart.com/docs/docs/intro
Templates (`npm create hyper-dart@latest`)	Templates
Keywords, regex and config	hyperdart.config.js
Publishing	Publish
npm registry API	REGISTRY-API.md
npm download counts API	download-counts.md
Entities (collection names and types for regexes)	Entities